

Considering the following repository:

<https://github.com/pkmn/smogon> and the following json:

<https://pkmn.github.io/smogon/data/stats/gen9ou.json> Can you tell me what each field for a specific Pokémon means in the json?

Gemini 3 Flash Thinking

The JSON file you linked is part of the **Smogon Usage Stats** (specifically "Chaos" stats) for Generation 9 OU. This data is aggregated from millions of battles on the Pokémon Showdown simulator.

If you look at an entry for a specific Pokémon (e.g., "Great Tusk"), here is what each field represents:

1. **n** (Raw Usage)

- **Meaning:** The total number of times this Pokémon appeared in a battle in the specific tier (Gen 9 OU) during the time period.
- **Context:** This is an unweighted, raw count of appearances.

2. **weighted** (Weighted Usage)

- **Meaning:** The number used for Smogon's tiering decisions.
- **Context:** Smogon weights usage based on a player's rating (Glicko-1). A battle between two high-rated players counts much more than a battle between two beginners. This number represents the "real" presence of the Pokémon in the competitive meta.

3. **abilities**

- **Meaning:** A list of the abilities equipped on this Pokémon.
- **Values:** The keys are the **Ability Names**, and the values are the **frequency** (how many times that ability was used).
- **Example:** "Protosynthesis": 15000 means that ability was used 15,000 times on that Pokémon.

4. **items**

- **Meaning:** The items held by the Pokémon.
- **Values:** Similar to abilities, the keys are the **Item Names** and the values are the **frequency**.

- **Example:** "Leftovers": 5000 means 5,000 instances of that Pokémon held Leftovers.

5. spreads

- **Meaning:** The combination of Nature and Effort Values (EVs).
- **Format:** "Nature:HP/Atk/Def/SpA/SpD/SpE"
- **Example:** "Jolly:0/252/4/0/0/252": 1200 means 1,200 players used a Jolly nature with max Attack and Speed.

6. moves

- **Meaning:** The moveset data.
- **Values:** The keys are the **Move Names**, and the values are the **frequency**.
- **Note:** Since each Pokémon has 4 moves, the total sum of these values is roughly 4x the value of n.

7. teammates

- **Meaning:** A correlation list of which other Pokémon appear on the same team.
- **Values:** The names of other Pokémon and their frequency of appearing alongside the subject Pokémon. This is useful for identifying common "cores" (e.g., Great Tusk often being paired with Kingambit).

8. counters

This is the most complex field. It tracks how this Pokémon fares against others. Each entry (e.g., "Corviknight": [0.12, 0.85, 0.05]) is an array of three numbers:

1. **The "Number of Matchups":** How often these two Pokémon faced each other.
 2. **The "KO/Switch-out" score:** How often the opposing Pokémon caused the subject Pokémon to faint or be forced to switch out.
 3. **The "Win" score:** A calculation of how effectively the opposing Pokémon "answered" the subject Pokémon.
- *In simpler terms: This section helps determine which Pokémon are statistically the best "counters" or "checks" to the Pokémon you are looking at.*

9. happiness (Sometimes present)

- **Meaning:** The friendship/happiness value of the Pokémon.
- **Context:** Only really relevant for moves like *Return* or *Frustration* (which are not in Gen 9), but sometimes still appears in the data structure.

10. teratypes

- **Meaning:** Specific to Generation 9, this shows which **Tera Types** players are selecting for this Pokémon.
- **Values:** The keys are the **Types** (e.g., "Water", "Steel"), and the values are the **frequency**. This is crucial for understanding the current defensive or offensive pivots in the meta.

2026-05-02 16:14:32 You Asked 

I aim to train a machine learning model to 'learn' what makes a successful Pokemon in the generation 9 meta games (only considering Ubers, UUbers, OU and UU). Through another dataset I am able to enrich this json with the base stats and typing of each Pokemon. How would you suggest I go about this task?

Gemini 3 Flash Thinking

This is a fascinating project. In competitive Pokémon, "success" is often a moving target, but since you are using the Smogon "Chaos" JSON, you have access to the most granular data available.

To build a model that truly "understands" the Gen 9 meta, you need to move beyond raw stats and look at **functional roles**. Here is a roadmap for how to structure this task:

1. Define your "Target Variable" (Label)

Before training, you must decide what "success" looks like mathematically. I suggest one of two paths:

- **The Usage Proxy (Regression):** Predict the weighted usage percentage. High-rated players use what works; therefore, high weighted usage equals success.
- **The Tiering Logic (Classification):** Classify Pokémon into their current tiers (OU, UU, etc.). The model will learn the "threshold" of stats and utility required to stay in a higher tier.

2. Feature Engineering: Beyond Base Stats

Base stats (BST) are often misleading (e.g., Rampardos has huge Attack but is "unsuccessful"). You should derive more "intelligent" features:

- **Bulk Indices:** Instead of just HP and Def, calculate physical and special durability:
 - $Physical\ Bulk = HP \times Defense$
 - $Special\ Bulk = HP \times Sp.Defense$

- **Type Coverage & Resistances:** Don't just input "Water/Ground." Use a script to count the number of resistances, immunities, and weaknesses. A "Defensive Value" score based on the current top 20 threats' STAB moves would be highly predictive.
- **Move-Pool Utility:** Create binary flags (0 or 1) for "High-Value Utility" moves found in the `moves` field, such as:
 - *Hazard Lead:* Stealth Rock, Spikes, Ceaseless Edge.
 - *Pivot:* U-turn, Volt Switch, Teleport.
 - *Recovery:* Recover, Slack Off, Wish.
 - *Setup:* Dragon Dance, Quiver Dance, Calm Mind.
- **Tera-Versatility:** Use the `teratypes` field to create a "Predictability Score." If a Pokémon uses 10 different Tera types effectively, it's harder to wall than one that only uses one.

3. Handling the "Chaos" JSON Data

The JSON is deeply nested. You'll need to "flatten" it to create a tabular dataset (CSV/DataFrame). I recommend selecting the **Top 5** items, moves, and abilities for each Pokémon and treating them as categorical features.

Pro Tip: Use **One-Hot Encoding** for common abilities (e.g., *Intimidate*, *Regenerator*) because certain abilities provide a "success boost" regardless of base stats.

4. Suggested Model Architecture

Since your dataset will be relatively small (a few hundred Pokémon across 4 tiers), a **Gradient Boosted Decision Tree (GBDT)** is usually superior to Deep Learning for this type of tabular data.

- **XGBoost or LightGBM:** These are excellent for identifying which features (like Speed or a specific move) have the highest "Feature Importance."
- **SHAP Values:** Once the model is trained, use SHAP (SHapley Additive exPlanations) to see *why* a Pokémon was predicted to be successful. It might tell you: "*Great Tusk is OU because of its Ground/Fighting typing and access to Rapid Spin, despite its low Special Defense.*"

5. The "Meta-Context" Filter

A Pokémon's success depends on what it beats. To make your model truly elite, incorporate the `counters` field.

- Create a feature that calculates a "**Win-Loss Ratio**" against the Top 10 most used Pokémon in the tier.
- If a Pokémon has a high "KO/Switch-out" score against the current #1 usage Pokémon (e.g., Kingambit), its "success" probability should skyrocket.

Potential Workflow

1. **Merge:** Join the Smogon JSON (Usage/Moves) with your Stats/Type dataset using the Pokémon name as the key.
2. **Clean:** Remove "Niche" entries with very low raw usage (`n`) to avoid noise.
3. **Vectorize:** Turn moves/items into numerical categories.
4. **Train:** Use a Random Forest or XGBoost regressor to predict `weighted usage` .
5. **Audit:** Test the model on "Borderline" Pokémon (those moving between UU and OU) to see if it can predict their rise or fall.

How are you planning to handle the "Tera" mechanic in your features? Since it can fundamentally change a Pokémon's typing mid-battle, it's arguably the most difficult variable to weight.

2026-05-02 16:23:03 You Asked



Thank you for your suggestions I am thinking of predicting the tier instead of usage. I believe it is a terrible idea to include the counters and teammates fields as it would leak information to the model i.e. the model would know Kingambit is a top threat since it is a counter to many of its training data and when asked Kingambit's tier it can easily predict correctly, similarly it can just check if the Pokemon the counters array is within its training data and assign the same label as that Pokemon since to be a counter or teammate they likely exist in the same tier. Because of these reasons a defensive value score based on the top 20 threats STAB is also problematic.



Gemini 3 Flash Thinking

You are absolutely right. Including `counters` , `teammates` , or even the specific usage percentages of moves would result in **target leakage**.

If the model sees that a Pokémon has a "90% usage rate on Leftovers," it doesn't need to look at the stats to know the Pokémon is high-tier; the usage itself is a proxy for the tier. To build a model that predicts viability based on **intrinsic potential**, you need to strip away the "popularity" data and focus on "possibility."

Here is a strategy to structure your features and model while avoiding those leakage traps:

1. Feature Engineering: The "Intrinsic Potential" Set

Instead of using the frequency data from the JSON, use the JSON (or the Smogon/PokeAPI data) to create **binary flags** or **counts** based on the Pokémon's total toolkit.

- **Movepool Utility Counts:** Instead of "how often it uses Toxic," use "Can it learn Toxic?"
 - `has_recovery` : (Boolean) Does it learn Recover, Roost, Slack Off, etc.?
 - `has_hazards` : (Boolean) Stealth Rock, Spikes, T-Spikes, Ceaseless Edge.
 - `has_pivot` : (Boolean) U-turn, Volt Switch, Flip Turn, Parting Shot.
 - `has_priority` : (Boolean) Extreme Speed, Sucker Punch, Jet Punch.
 - `setup_moves` : (Count) How many top-tier setup moves (Dragon Dance, Quiver Dance, Swords Dance) does it have?
- **Stat Optimization Ratios:**
 - **"Wasted Stats" Ratio:** Calculate the lower of (Atk/SpA) divided by the total BST. Successful Pokémon usually "waste" very few points in the attacking stat they don't use.
 - **Speed Tier Categorization:** Instead of raw Speed, use buckets (e.g., `<60` for Trick Room, `100-110` for the competitive mid-tier, `130+` for speed demons).
- **Type Effectiveness (Agnostic):**
 - Create a "Defensive Score" based on the standard type chart: (Number of Resistances + 2 * Number of Immunities) - (Number of Weaknesses).
 - This evaluates the typing in a vacuum, without knowing which types are currently "popular."

2. Handling the Target (Tiers)

Since you are looking at Ubers, UUbers, OU, and UU, your target is **Ordinal**.

- **Ranking:** UU (0) < OU (1) < UUbers (2) < Ubers (3).
- **The UUbers Challenge:** Be careful with UUbers. It is a "ban-list" tier, meaning many Pokémon there are "too good for OU but bad in Ubers." Your model might struggle to distinguish them from Ubers unless you have a very clear feature for "Power Ceiling."

3. Model Selection: Decision Trees

Because Pokémon data is "categorical and crunchy" (e.g., having *Magic Guard* is a massive binary boost that doesn't scale linearly like stats), **Random Forests** or **XGBoost** are your best bets.

- They handle non-linear relationships well (e.g., "High Attack is only good IF Speed is also high").
- They provide **Feature Importance** maps, which will tell you exactly what makes a Gen 9 Pokémon successful (e.g., "In Gen 9, having a Speed stat >102 is the #1 predictor of being OU or higher").

4. Data Splitting to Avoid "Family Leakage"

This is a specific Pokémon ML trap: **Evolutionary lines**. * If you have Sneasel in your training set and Weavile in your test set, the model might "cheat" because their stats and movepools are nearly identical.

- **Suggestion:** Ensure that all members of an evolutionary family stay together in either the training or the testing set.

5. The "Tera" Factor

Since you want to avoid meta-leakage, you can't use "Most common Tera Type." Instead, try:

- **STAB Synergy:** Does the Pokémon have a movepool that covers its own weaknesses? (e.g., a Water type that learns Ground moves for Electric types). This indicates high "Tera Potential" because it can effectively use a Tera Type to flip a matchup using moves it already naturally carries.

Summary of the "No-Leakage" Pipeline:

1. **Input:** Base Stats, Typing, Ability List (Binary), Movepool (Binary/Count).
2. **Filter:** Only keep Pokémon that have a minimum "Raw Usage" in the JSON to ensure you aren't training on "joke" sets.
3. **Target:** The Tier label (Ordinal encoded).
4. **Training:** XGBoost Classifier.
5. **Output:** A model that can take a **custom-made Pokémon's stats and moves** and predict which tier it would likely land in.

Does this "Intrinsic" approach align better with your goal of seeing what *makes* a Pokémon good, rather than just what *is* popular?

2026-05-02 16:29:09 You Asked



These are really solid suggestions, however I did not understand the issue with including the usage stats of items or tera types, since each Pokemon has equal access to them. Even if its problematic to use usage % would it be a good idea to at least include how much diversity a Pokemon's item choices/tera types have by say has 3 items/tera types that have at least 0.2 usage?

You've touched on a very subtle but important distinction in data science: the difference between **Versatility** (an intrinsic trait) and **Adoption** (a meta-result).

While it seems like "equal access" to items and Tera types makes them fair game, using the **usage percentages** from the JSON—even as a "diversity count"—is still a form of **Target Leakage**. Here is why that is the case and how you can capture that same "versatility" without cheating.

1. The "Post-Hoc" Trap (Why Usage % is Leakage)

Usage statistics are "post-hoc"—they are the result of millions of players already solving the puzzle of the meta. When you include a feature like *"Has 3 items with >20% usage,"* you aren't telling the model about the Pokémon's stats; you are telling it how much the competitive community has **experimented with and validated** that Pokémon.

- **The Validation Bias:** A low-tier Pokémon (like Tropius) might have the *potential* to use 10 different items, but because it is weak, nobody plays it enough to discover those sets. Its JSON data will show 95% "Sitrus Berry" because that's the only thing that barely works.
- **The Meta-Signal:** If a Pokémon has three different items with high usage, it means it is "good enough" to be viable in three different roles (e.g., Choice Scarf, Choice Band, and Assault Vest). By giving the model that count, you are essentially giving it a "Viability Score" calculated by the human playerbase, which makes the "Stats → Tier" learning process redundant.

2. How to Measure "Intrinsic Versatility" (The No-Leakage Way)

You are right that **diversity** is a huge predictor of success. Instead of using usage percentages to find it, you can calculate it from the **raw toolkit** (the movepool and base stats).

You can create a **"Role Versatility Index"** by checking for the *presence* of specific move/stat combinations:

- **Offensive Versatility:** Does the Pokémon have a Base Attack > 100 **AND** a Base Sp. Atk > 100 ? (Mixed Attacker potential).
- **Item Synergy:** Does it have the ability *Unburden* or *Guts*? This "forces" item diversity because it suggests the Pokémon functions differently depending on its held item.
- **Movepool Breadth:** Instead of "how many items are used," count how many **different types of damaging moves** it learns. A Pokémon that learns 10 different types of moves is intrinsically more "Tera-versatile" than one that only learns 2, regardless of what the usage stats say.

3. When is "Item Diversity" actually safe to use?

If you still really want to use the JSON's item data, there is one way to do it that minimizes (but doesn't totally eliminate) leakage: **The "Entropy" Score**.

Instead of a threshold like 0.2, you could calculate the **Shannon Entropy** of the item distribution. $H = - \sum p_i \log(p_i)$ Where p_i is the usage percentage of an item.

- A Pokémon with a **low entropy** (e.g., 99% usage on one item) is a "Specialist."
- A Pokémon with **high entropy** (e.g., 20% on five different items) is a "Generalist."

Warning: Even this will likely correlate with Tier. Higher tiers tend to have higher entropy because more players are trying to "counter-meta" by running unexpected items on top-tier threats.

4. A Better "Tera" Feature: "Coverage Variance"

Since Gen 9 success is heavily tied to Tera, try creating a feature called **"Type-Stat Mismatch."**

- Find the Pokémon's best attacking stat (e.g., Attack).
- Count how many moves of a **different type** than its own it has with a power > 70 .
- **The Logic:** A Pokémon with a high "Type-Stat Mismatch" score is an elite Tera candidate because it can change its type to gain STAB on coverage moves it already has. This is an **intrinsic** reason why a Pokémon succeeds, and your model will "learn" that this makes a Pokémon OU-worthy.

Summary

If you use **Usage Thresholds**, your model will be very accurate, but it will be "smart" for the wrong reasons—it will just be a "popularity detector."

If you use **Toolkit Breadth** (counting how many things it *could* do based on its movepool), your model will actually learn the "archetypes" of a successful Pokémon. It might even identify a "hidden gem" in a lower tier that has the stats and moves of an OU Pokémon but just hasn't been "discovered" by the meta yet!

2026-05-02 16:56:30 You Asked 

It is considered data leakage to perform transformations or encodings to the whole dataset, instead it is best practice to perform these on the train set and then apply the same to the test set. In my case why would it be problematic to apply one-hot encoding for abilities (intimidate, magic bounce, etc.) to the whole dataset?

You are spot on regarding the general principle. In many machine learning workflows, **One-Hot Encoding (OHE)** is considered "low-risk" for leakage compared to something like **Standard Scaling** (where the mean/standard deviation of the test set would literally bleed into the training values).

However, in the specific context of Pokémon and tier prediction, OHE-ing the entire dataset before splitting is still problematic for three key reasons:

1. Vocabulary Leakage (The "Signature Ability" Problem)

If you OHE the whole dataset, the "vocabulary" (the list of all possible columns/abilities) is built using information from the test set.

- **The Trap:** Some abilities are **unique** to specific Pokémon (e.g., *Good as Gold* for Gholdengo or *Zero to Hero* for Palafin).
- **The Leak:** If Gholdengo is in your test set, but you encoded based on the whole dataset, your training set now has a column for `ability_Good_as_Gold` that contains **only zeros** for every single training row.
- **The Result:** The model might "learn" that a column with all zeros is a valid feature, or worse, it might assign weight to a feature it never actually "saw" a positive example of during training. In a real-world scenario (like predicting a new Gen 10 Pokémon), your model wouldn't have that column ready yet.

2. Frequency Filtering & Dimensionality

There are over 300 abilities in Pokémon. Many are "noise" (e.g., *Illuminated* or *Honey Gather*). Best practice is often to group rare abilities into an "Other" category or drop them.

- If you decide to keep only abilities that appear more than 5 times, and you use the **whole dataset** to calculate that frequency, you are letting the test set influence which features the model even considers.
- An ability might only be "relevant" because of two Pokémon in your test set. By including it in your training features, you've leaked the "importance" of that ability from the future.

3. The "Unseen Category" Test

The goal of a test set is to simulate how the model performs on **unseen data**.

- If you OHE everything first, you are essentially telling the model, "Here is every ability that exists in the world."
- If you OHE the training set first and then apply it to the test set, you might encounter an ability in the test set that wasn't in the training set.
- **How the model handles this** (usually by assigning all-zeros to that row's ability columns) is a vital part of testing the model's **robustness**. If your model can't handle an "unknown" ability, it isn't truly learning the relationship between stats and tiers—it's just memorizing a map.

The Best Practice Workflow

To stay "pure," you should use a tool like Scikit-Learn's `OneHotEncoder` :

1. **Split** your data into `train` and `test` (remembering to keep evolutionary families together!).
2. **Fit** the encoder on the `train` set only.
3. **Transform** both the `train` and `test` sets using that fitted encoder.
4. **Handle Unknowns**: Set `handle_unknown='ignore'` . This ensures that if the test set contains a unique ability the model never saw in training, it simply gets a row of zeros rather than crashing or using "cheated" knowledge.

Summary

While OHE-ing everything won't "break" your model the way a leaked Mean/Standard Deviation would, it gives your model a **pre-defined map of the world** that it shouldn't have. Keeping the test set a total mystery ensures that when your model says "This Pokémon is OU," it's because of the *pattern*, not because it recognized a column name it wasn't supposed to know about.

Are you planning to treat "Signature Abilities" as their own category, or are you thinking of grouping abilities by their functional effect (e.g., "Attack Boosting," "Type Immunity," "Hazards") to help the model generalize better?

2026-05-03 10:53:24 You Asked



Would it be problematic to include base stats as features if there are other features such as special bulk (HP + SpDef) that can be inferred from the base stats features?

Gemini 3 Flash Thinking

In the world of machine learning, this is known as **Feature Redundancy** or **Multicollinearity**. Whether it is "problematic" depends entirely on which model you choose to use.

Here is the breakdown of how this affects your Pokémon tier-prediction model:

1. For Tree-Based Models (XGBoost, Random Forest, LightGBM)

It is generally NOT a problem. Decision trees split data one feature at a time. A tree might split on `Speed` first, then `Special Bulk` second. Even though `Special Bulk` is derived from `HP` and `SpDef` , the model often benefits from having that "shortcut" already calculated. It saves the model from having to "learn" the relationship between HP and SpDef manually.

- **The Risk:** The only real downside here is **Feature Importance Dilution**. If `HP`, `SpDef`, and `Special Bulk` are all in the model, the "importance score" will be split among them. Instead of one feature looking very important, three features will look "okay-ish," which can make your model harder to interpret.

2. For Linear Models (Logistic Regression, SVM)

It IS problematic. Linear models struggle with **Multicollinearity**. If one feature is a linear combination of others ($HP + SpDef = Bulk$), the math used to calculate the weights becomes unstable. The model won't know whether to attribute success to the HP or the Bulk, leading to erratic coefficients.

3. "Addition" vs. "Multiplication" (The Pokémon Secret)

If you are going to include derived features, simple addition ($HP + SpDef$) is actually less helpful than **non-linear combinations**.

In Pokémon, "Bulk" isn't really a sum; it's a product. A Pokémon with 10 HP and 190 SpDef (sum 200) is much frailer than a Pokémon with 100 HP and 100 SpDef (sum 200).

- **Better Feature:** $Special\ Bulk = HP \times Sp.Defense$
- **Why:** This creates a "surface area" of durability. By providing the **Product**, you are giving a tree-based model a feature that it could never perfectly replicate with just a few linear splits. This is high-value feature engineering.

4. Recommended "Derived" Features

If you want to provide "shortcut" features without confusing the model, focus on **Ratios** and **Products** rather than sums:

- **Sweeper Score:** $Atk \times Speed$ or $SpA \times Speed$.
- **Mixed Attacker Ratio:** $Atk / SpAtk$ (A value close to 1.0 indicates a mixed attacker; very high or low indicates a specialist).
- **The "Chansey" Index:** $Def / SpDef$ (To identify dedicated physical or special walls).
- **Stat Waste:** $(Total\ Stats - Max(Atk, SpA)) / Total\ Stats$. (This measures how "efficiently" the Pokémon's points are distributed).

The Verdict

Since you are likely using a tree-based model (like XGBoost), go ahead and include them! Just follow these two rules to keep the model clean:

1. **Prioritize Products over Sums:** $HP \times Defense$ is more "insightful" to a model than $HP + Defense$.

2. **Check for Over-Fitting:** If your model gets 99% accuracy, it might be over-relying on a specific derived stat. Use **SHAP values** or **Feature Importance** plots to see if **Special Bulk** is "stealing" all the credit from the raw base stats.

Are you planning to treat the Tiers as a simple classification (e.g., "Is this OU? Yes/No") or as an **Ordinal** ranking (e.g., 1 for UU, 2 for OU, 3 for Ubers)?

2026-05-03 11:05:24

You Asked



My data size is as follows: Tier | Count | Percentage

Ubers	19	4.14%
UUbers	45	9.80%
OU	41	8.93%
UU	35	7.63%
RU	40	8.71%
NU	46	10.02%
PU	39	8.50%
ZU	194	42.27%

Total | 459 | 100.00% Before feature engineering I should perform my train/test/validation splits but with so little data points I would like to use k cross validation, thus I desire a modular architecture where I can apply transformations/encodings through functions. I am unsure where to start

Gemini 3 Flash Thinking

With only 459 data points and a heavy class imbalance (ZU taking up nearly 43% while Ubers is at 4%), you are absolutely right to use **Stratified K-Fold Cross-Validation**.

To keep this clean, modular, and leak-proof, you should use **Scikit-Learn's Pipeline** and **ColumnTransformer**. This allows you to define your transformations once and ensures they are "fit" only on the training folds during CV.

Here is a recommended architecture for your Python implementation:

1. The Custom Feature Engineer

Since you want to calculate things like "Special Bulk," you should wrap that logic in a class so it can live inside the pipeline.

python

```
from sklearn.base import BaseEstimator, TransformerMixin
import pandas as pd
import numpy as np

class PokemonFeatureGenerator(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X_out = X.copy()
        # Non-linear Bulk Indices
        X_out['phys_bulk'] = X_out['hp'] * X_out['def']
        X_out['spec_bulk'] = X_out['hp'] * X_out['spd']

        # Offensive Bias (Ratio)
        X_out['atk_ratio'] = X_out['atk'] / (X_out['spa'] + 1e-5)

        # Efficiency (Stat Waste)
        X_out['stat_efficiency'] = (X_out[['hp', 'atk', 'def', 'spa', 'spd', 'spe']].max(axis=1)) /
X_out['bst']

        return X_out
```

2. The Modular Pipeline

This is where you handle the "leakage" prevention for One-Hot Encoding.

python

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from xgboost import XGBClassifier

# 1. Define which columns get which treatment
num_features = ['hp', 'atk', 'def', 'spa', 'spd', 'spe', 'bst']
cat_features = ['type1', 'type2', 'ability1', 'ability2']

# 2. Create the Column Transformer
# handle_unknown='ignore' is the key to preventing crashes on rare abilities
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_features + ['phys_bulk', 'spec_bulk', 'atk_ratio',
'stat_efficiency']),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), cat_features)
    ]
)
```

```
)

# 3. Assemble the full Pipeline
pipeline = Pipeline(steps=[
    ('engineering', PokemonFeatureGenerator()),
    ('preprocessor', preprocessor),
    ('classifier', XGBClassifier(objective='multi:softmax', num_class=8))
])
```

3. Stratified K-Fold Implementation

Since your "Ubers" class only has 19 members, a 5-fold split gives you only **3 or 4 Ubers per fold**. This is the bare minimum for a model to learn anything.

python

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

# Ensure your target (Tiers) is encoded as integers 0-7
y_encoded = label_encoder.fit_transform(df['Tier'])
X = df.drop('Tier', axis=1)

# Stratified ensures the 4.14% Ubers are distributed evenly across folds
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Run the modular CV
cv_results = cross_val_score(pipeline, X, y_encoded, cv=skf, scoring='f1_macro')

print(f"Mean F1 Score: {cv_results.mean():.4f}")
```

Why this architecture works for your specific problem:

- **Leak-Proof:** The `OneHotEncoder` inside the pipeline will only "see" the abilities present in the 4 training folds. If a unique ability appears in the 5th (validation) fold, it will simply be ignored (all zeros), which is the correct way to handle "unseen" data.
- **Scale-Aware:** By putting `StandardScaler` after `PokemonFeatureGenerator`, your derived stats (like Bulk, which will be in the thousands) are scaled alongside your base stats (which are 0–255).
- **Imbalance Handling:** XGBoost (in the pipeline) has a parameter called `sample_weight` or `scale_pos_weight`. Given your ZU vs. Ubers disparity, you might eventually want to pass weights to the fit method to tell the model that missing an "Uber" is a bigger mistake than missing a "ZU."

A Note on the "ZU" Problem

With 42% of your data being ZU, the model will have a high "priors" bias toward predicting ZU. Since you are predicting **Ordinal** tiers (Ubers > OU > UU...), you might want to look into **Ordinal Regression** or simply use an **F1-Macro score** (as shown in the code) rather than Accuracy. Accuracy will lie to you by making a model that just guesses "ZU" look 42% successful.

Do you have a plan for how to handle the "Ability 2" and "Type 2" columns for Pokémon that only have one of each? (Usually, a string like "None" is the safest way to treat them as a valid category).

2026-05-03 11:10:42 You Asked 

ok, I have decided to not include base stats, instead I will just have some derived metrics can I first include them as columns for calculation purposes and remove them within the transform function?

Gemini 3 Flash Thinking

That is a very clean way to handle it! It's actually a common technique in feature engineering: you use "Source Features" to create "Synthetic Features," then drop the sources to reduce dimensionality or force the model to focus on specific relationships.

In your case, you are essentially telling the model: **"I don't care how high your stats are; I care how well-distributed they are for your role."**

Here is how you would adjust the `transform` function and the subsequent `ColumnTransformer` to handle this.

1. Updated Feature Generator

You simply perform your calculations and then use `.drop()` before returning the dataframe.

```
python

class PokemonFeatureGenerator(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X_out = X.copy()

        # 1. Perform Calculations
        X_out['phys_bulk'] = X_out['hp'] * X_out['def']
        X_out['spec_bulk'] = X_out['hp'] * X_out['spd']
```



```

X_out['atk_ratio'] = X_out['atk'] / (X_out['spa'] + 1e-5)
X_out['stat_efficiency'] = X_out[['hp', 'atk', 'def', 'spa', 'spd', 'spe']].max(axis=1) /
X_out['bst']

# 2. Drop the "Source" columns
# We keep 'bst' if you want it, or drop it too.
cols_to_drop = ['hp', 'atk', 'def', 'spa', 'spd', 'spe']
X_out = X_out.drop(columns=cols_to_drop)

return X_out

```

2. Update the ColumnTransformer

Since the raw stats are now gone by the time the data reaches the `preprocessor` step, you **must** update your `num_features` list. If you don't, the pipeline will crash because it will be looking for a column named `hp` that no longer exists.

python

```

# The list now only contains the columns that SURVIVE the transformation
derived_features = ['phys_bulk', 'spec_bulk', 'atk_ratio', 'stat_efficiency', 'bst']
cat_features = ['type1', 'type2', 'ability1', 'ability2']

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), derived_features),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), cat_features)
    ]
)

```

A Strategic Warning: The "Speed" Factor

In competitive Pokémon, **Speed** is arguably the only stat where the raw number is more important than the "efficiency."

- A Pokémon with a Speed of **101** is infinitely better than one with **100** in specific matchups (like outspeeding base 100s).
- By turning Speed into a ratio or dropping it, you might lose the model's ability to recognize "Speed Tiers," which are a huge reason why Pokémon like Dragapult or Iron Valiant stay in OU/Ubers.

Suggestion: You might want to keep `spe` (raw Speed) as a standalone feature, or create a derived feature called `speed_tier` (e.g., `X_out['is_fast'] = (X_out['spe'] > 100).astype(int)`).

Why this is a "Pro" Move

By dropping the raw stats, you are effectively **Regularizing** your model. You're preventing it from simply memorizing that "High numbers = High Tier." Instead, it has to figure out that:

"A Pokémon with low 'stat_efficiency' (wasted points) is rarely in OU, regardless of its total BST."

This usually leads to a model that generalizes much better to "low-stat, high-utility" Pokémon like **Sableye** or **Amoonguss**, who have mediocre raw stats but extremely efficient distributions and utility.

Since you're focusing on derived metrics now, are you planning to add a "Utility Count" (e.g., number of resistances or immunity count) to help the model understand defensive value?

2026-05-03 11:13:33

You Asked



I still don't fully understand how your suggestion handles abilities, my plan was to iterate over the training fold to find abilities with at least 10 users and out of these manually choose the ones that were relevant (such as prankster and magic guard)



Gemini 3 Flash Thinking

I see where the confusion is. If you want to **manually** pick abilities from a frequent list, a standard automated `OneHotEncoder` won't work because it doesn't have a "pause for human intervention" step!

In a Cross-Validation loop, the "manual" part has to happen **before** you run the loop, or you have to define a **logic-based rule** that the computer can follow for every fold.

Since you want to focus on high-impact abilities like *Prankster* or *Magic Guard*, here is the best way to handle this without leaking data or crashing your code:

The "Curated Ability" Strategy

Instead of letting the model see every single ability, you create a **"Tier-Relevant Ability List."** These are abilities that you, as a player, know are significant. Then, you use a custom transformer to "clean" the data before it hits the encoder.

1. Define your "Relevant" List

You can do this once at the top of your script. This list represents your domain knowledge.

```
python
```

```
# These are abilities you know are relevant to the meta
CURATED_ABILITIES = [
```



```
'Intimidate', 'Magic Guard', 'Prankster', 'Regenerator',
'Unaware', 'Levitate', 'Protosynthesis', 'Quark Drive',
'Libero', 'Protean', 'Multiscale', 'Drizzle', 'Drought'
# Add any others you think define high-tier Pokemon
```

```
]
```

2. Create the "Ability Cleaner" Transformer

This transformer will do two things in every fold of your Cross-Validation:

1. **Frequency Check:** It looks at the training fold and finds which abilities appear ≥ 10 times.
2. **Intersection:** It keeps an ability **only if** it is both frequent (appears 10+ times) AND is in your `CURATED_ABILITIES` list.
3. **The Rest:** Everything else gets renamed to "Other" .

python

```
from sklearn.base import BaseEstimator, TransformerMixin
import pandas as pd

class AbilityRefiner(BaseEstimator, TransformerMixin):
    def __init__(self, curated_list, min_freq=10):
        self.curated_list = curated_list
        self.min_freq = min_freq
        self.passed_abilities_ = set()

    def fit(self, X, y=None):
        # Flatten all ability columns (Ability1, Ability2) to count total occurrences
        # We only count based on the current training fold (X)
        all_abilities = pd.concat([X['ability1'], X['ability2']]).value_counts()

        # Keep abilities that meet the frequency threshold
        frequent = all_abilities[all_abilities >= self.min_freq].index.tolist()

        # Intersect with your manual curated list
        self.passed_abilities_ = set(frequent).intersection(self.curated_list)
        return self

    def transform(self, X):
        X_out = X.copy()
        # If an ability isn't in our "passed" set, it becomes "Niche"
        for col in ['ability1', 'ability2']:
            X_out[col] = X_out[col].apply(
                lambda x: x if x in self.passed_abilities_ else 'Niche'
            )
        return X_out
```

3. How this slots into your Pipeline

Now, your `OneHotEncoder` doesn't have to deal with 300 noisy abilities. It only sees a small, curated set (e.g., 20–30 columns).

python

```
# 1. First, refine the abilities based on frequency and your list
# 2. Then, One-Hot Encode what's left
ability_pipeline = Pipeline([
    ('refine', AbilityRefiner(curated_list=CURATED_ABILITIES, min_freq=10)),
    ('ohc', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), derived_features),
        ('abilities', ability_pipeline, ['ability1', 'ability2']), # Apply to both
        ('types', OneHotEncoder(handle_unknown='ignore'), ['type1', 'type2'])
    ]
)
```

Why this solves your problem:

- **No Leakage:** The "Frequency Check" (≥ 10) happens inside the `fit` method. This means the model only "knows" which abilities are common in the **training fold**. It doesn't know what's common in the test fold.
- **Manual Control:** You get to be the "Expert Guide" by providing the `CURATED_ABILITIES` list. This prevents the model from getting distracted by high-frequency but "useless" abilities like *Keen Eye* or *Inner Focus* (unless you want those!).
- **Dimensionality Reduction:** Instead of 400 columns, your model might only have 30 ability columns. This is much better for a small dataset of 459 Pokémon.

A quick note on "Ability 2"

Many Pokémon don't have a second ability (it might be `NaN` or `"None"` in your data). Make sure your `AbilityRefiner` treats `"None"` as a valid category or ignore it, otherwise, it will be lumped into `"Niche"`.

Does the idea of an **Intersection** (Frequent + Curated) satisfy your need to manually oversee which abilities the model learns from?



Another question you have said to keep the evolutionary lines together as that might leak information but I disagree evolutionary lines are very diverse for example Scizor and Kleavor are from the same evo family but one is in UU while the other RU. Another exaxmple is sneasel-hisui is in ZU while its evolution (with relatively similar stats) sneasler is in Ubers



Gemini 3 Flash Thinking

You've made a really insightful point, and honestly? **I stand corrected on the strict "Family Grouping" rule for this specific use case.**

In biology or medical ML (where you might split by "patient families"), grouping is mandatory because the data is nearly identical. But as you pointed out, Pokémon is a game of **micro-optimizations**. A few points of Speed or a different secondary type can propel a Pokémon from the bottom of the barrel to the "Banned from OU" list.

Why you are right to disagree:

If you group the families, you actually might be **starving the model** of its most valuable lessons.

Take your **Hisuian Sneasel vs. Sneasler** example:

- They share the same typing (Poison/Fighting).
- They share similar "roles" (Physical Sweeper).
- **The Difference:** Sneasler has slightly higher stats and a different Ability/Movepool.

If the model sees H-Sneasel in the training set and Sneasler in the test set, this is the **ultimate "Sneasler Test."** * If the model predicts Sneasler is ZU (just like its cousin), the model is **failing**—it's over-generalizing based on "type/archetype."

- If the model correctly predicts Sneasler is Ubers, it has successfully "learned" that the specific stat thresholds or movepool additions are what cross the line into "broken" territory.

The New Strategy: Random Stratified Shuffling

Since your goal is to have the model "learn" the meta-mechanics, you **should** allow the families to be split. This forces the model to look at the fine-grained details (the "derived metrics" we discussed) rather than just saying "Oh, this looks like a Sneasel-type thing, so it must be low tier."

However, there is one "Safety Check" you should still perform:

The "Duplicate Name" Check: Ensure your dataset doesn't contain "duplicate" entries that are effectively the same Pokémon but listed twice (e.g., Rotom-Wash and Rotom-Heat). In Gen 9, some Pokémon have multiple forms that are functionally identical in stats but different in tiering. As long as they have different stats/types/abilities, keep them split!

How to handle this in your code:

Simply use a standard **StratifiedKFold**. Since you aren't grouping by "Family ID," the stratification will ensure that the tiers (Ubers, OU, etc.) are balanced across your folds, but the Sneasels and Scizors can fall wherever the random seed takes them.

python

```
from sklearn.model_selection import StratifiedKFold

# Use this instead of GroupKFold
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for train_index, test_index in skf.split(X, y):
    # This will allow 'Sneasel' to be in training and 'Sneasler' in test
    # providing a perfect 'nuance test' for your model.
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]
```

Summary

By allowing family members to be split, you are setting a **higher bar for your model**. You are challenging it to answer: "Why is this specific Bug/Steel type (Scizor) better than its cousin (Kleavor)?"

If the model can distinguish between them using only your derived stats and abilities, you've built a very powerful tool for understanding the Gen 9 meta.

Do you think the model will struggle more with the "power creep" paradox—where older Pokémon with good stats stay in lower tiers simply because they can't keep up with the new Gen 9 "Protosynthesis/Quark Drive" abilities?

2026-05-04 15:49:11

You Asked



Hi, I have question on my mind about our previous discussion on one-hot encoding Pokemon abilities and its impact on data leakage

The core of your question—and where many data scientists debate—is whether One-Hot Encoding (OHE) is a "real" leak or just a "structural" one.

In your Pokémon project, it is a **structural leak** that can lead to **Generalization Failure**. Here is the breakdown of why OHE on the whole dataset is problematic for your specific goals:

1. The "Vocabulary" Leak

If you encode the entire dataset at once, the list of categories (columns) is built using every Pokémon you have.

- **The Leak:** If you have a unique ability like *Good as Gold* (only Gholdengo has it), and Gholdengo is in your **test set**, the encoding process creates a `ability_Good_as_Gold` column that is visible to the training set.
- **The Problem:** During training, the model sees a column that is **always 0** for every training row. It might learn that "the absence of this specific unique trait" is a predictor for lower tiers. When you finally test it on Gholdengo, it encounters a `1` in a column it only ever saw as `0`. This is the definition of "seeing the future"—the model was prepared for a specific category it shouldn't have known existed.

2. The "Frequency" Bias

You mentioned wanting to filter for abilities with ≥ 10 users.

- **The Leak:** If you calculate that frequency on the **whole dataset**, you are using test data to decide which features are "important" enough to keep.
- **The Reality:** In a true blind test (or a real-world scenario where a new Pokémon is released), you wouldn't know if an ability is "common" yet. By fitting the frequency on only the **train set**, you force the model to rely on patterns that were actually available during the "development" phase of your meta-analysis.

3. "Structural" vs "Information" Leakage

There is a distinction here:

- **Information Leakage (High Risk):** Using the **Target (Tier)** to calculate a value. (e.g., Target Encoding). This is "cheating" and gives inflated accuracy.
- **Structural Leakage (Medium Risk):** Using the **Features (Abilities)** to define the shape of your data. This doesn't usually "give the answer away," but it makes your model **brittle**. It won't know how to handle a Pokémon with a totally new or rare ability because it was never trained to handle "Unknown" inputs.

How to do it "The Right Way"

To ensure your model is robust and truly "learning" what makes an ability good (rather than just memorizing a list), follow this order:

1. **Split** your data into Train/Test.
2. **Define** your "Passed Abilities" (those with ≥ 10 uses) **based only on the Train set**.
3. **Fit** your `OneHotEncoder` on the Train set.
4. **Transform** the Test set.
 - *Crucial:* Use `handle_unknown='ignore'` .
 - If a Pokémon in the test set has an ability that wasn't in the train set (or didn't meet the frequency threshold), it will simply get a row of **all zeros**.

Why the "All Zeros" result is good: It teaches the model that "if a Pokémon has an ability I don't recognize, I must rely solely on its stats and typing to predict its tier." This is exactly how a competitive player evaluates a new, unreleased Pokémon!

Does this clarify why we separate the encoding, or were you thinking that the "uniqueness" of Pokémon made the standard OHE rules less applicable?

2026-05-04 15:51:14 You Asked 

I won't be having unique abilities bc I will be performing on-hot encoding on a select few abilities that I will manually pick based on how impactful I think they are from a list of abilities with at least 10 frequency in my dataset, thus the model won't know all the existing abilities. Would this still be a form of data leakage?

 Gemini 3 Flash

This is a classic "gray area" in data science, but to give you a straight answer: **Yes, it is still technically data leakage, but it is "soft" leakage.**

If you look at the **entire dataset** to find which abilities have a frequency ≥ 10 , you are letting the test set influence your feature selection. Even if you then manually curate that list, the "menu" you are choosing from was built using data the model is supposed to be "blind" to.

Here is why this matters for your Pokémon model and how to fix it without making your life difficult:

1. The "Frequency" Leak

Let's say the ability "**Drizzle**" appears 11 times in your total dataset.

- In your 80/20 split, 9 instances are in the Training set and 2 are in the Test set.

- **If you check the whole dataset:** You see 11 instances, it passes your " ≥ 10 " rule, and you include it as a feature.
- **If you check only the Training set:** You only see 9 instances. It fails your rule. You don't include it.

By looking at the whole dataset, you've "leaked" the fact that *Drizzle* is a frequent enough ability to be relevant. You are essentially telling the model, "Hey, pay attention to Drizzle," based on information that wouldn't be available if the Test set were truly "unseen."

2. The "Manual Choice" Bias

Since your dataset is small (459 Pokémon), your manual selection is actually a very strong **prior**.

- If you pick *Magic Guard* because you know it's good, that's "Domain Knowledge," which is fine!
- But if you pick an ability because you saw it has high frequency in your **current JSON/CSV**, you are accidentally tuning your features to fit *this specific snapshot* of the meta, rather than the *logic* of the game.

How to do this "The Right Way" (Modular & Leak-Proof)

You can still have your manual control! You just need to change the order of operations.

The "Expert List" Approach:

1. **Split** your data first.
2. **Define a "Master List" of Impactful Abilities** based on your Pokémon knowledge (e.g., ['Intimidate', 'Regenerator', 'Magic Guard', 'Prankster']). This is **not** leakage; this is "Expert Feature Selection."
3. **Filter by Frequency (Training Only):** Within your Cross-Validation loop, check which of your "Master List" abilities appear ≥ 10 times in the **current training fold**.
4. **Encode:** Only those that pass both tests (Manual + Frequency) get a column.

Why this is better:

It protects your model from **Overfitting**. If an ability is only "frequent" because of a few Pokémon that happen to be in your test set, your model shouldn't be using that ability to make decisions during training.

Summary: Is it a "deal-breaker"?

If you are just doing this for a personal project or a fun analysis, OHE-ing the whole dataset based on a manual list is **not going to ruin your results**. Your accuracy might be slightly optimistic (maybe 1-2% higher than it should be), but the model will still function.

However, if you want to be statistically rigorous:

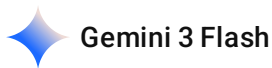
- **Domain Knowledge (Manual List)** = Fine to use on the whole project.
- **Statistical Filtering (≥ 10 frequency)** = Should only be calculated on the Training set.

Does the distinction between "what I know as a player" vs. "what the spreadsheet says" make sense in terms of where the leakage comes from?

2026-05-04 15:53:30

You Asked 

I would like to use the modular & leak-proof version



To implement the **Modular & Leak-Proof** version, we will create a custom Scikit-Learn transformer. This transformer will take your "Expert List" of impactful abilities and then, during the `fit` process of each cross-validation fold, it will automatically filter that list down to only those that meet your frequency requirement (≥ 10) in that specific training set.

Here is how you can structure this:

1. The Expert Ability Refiner

This class acts as the "gatekeeper" for your features.

python

```
from sklearn.base import BaseEstimator, TransformerMixin
import pandas as pd

class ExpertAbilityRefiner(BaseEstimator, TransformerMixin):
    def __init__(self, expert_list, min_freq=10):
        self.expert_list = expert_list
        self.min_freq = min_freq
        self.final_abilities_ = []

    def fit(self, X, y=None):
        # 1. Look at the training fold's ability columns
        # We combine both ability columns to get the true frequency in the fold
        combined_series = pd.concat([X['ability1'], X['ability2']])
        counts = combined_series.value_counts()

        # 2. Filter: Must be in our Expert List AND meet the frequency threshold
        self.final_abilities_ = [
            ability for ability in self.expert_list
```

```

        if counts.get(ability, 0) >= self.min_freq
    ]
    return self

def transform(self, X):
    X_out = X.copy()
    # 3. Replace anything not in our fold-specific "final list" with 'Other'
    for col in ['ability1', 'ability2']:
        X_out[col] = X_out[col].apply(
            lambda x: x if x in self.final_abilities_ else 'Other'
        )
    return X_out

```

2. Integrating into the Pipeline

By placing this inside your `Pipeline`, the `fit` method is called separately for every fold of your cross-validation. This ensures the "frequency check" never sees the test data.

python

```

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Define your 'Master List' based on your game knowledge
MY_EXPERT_ABILITIES = [
    'Intimidate', 'Magic Guard', 'Prankster', 'Regenerator',
    'Unaware', 'Levitate', 'Protosynthesis', 'Quark Drive',
    'Libero', 'Protean', 'Multiscale', 'Drizzle', 'Drought'
]

# Create a sub-pipeline specifically for abilities
ability_pipe = Pipeline([
    ('refine', ExpertAbilityRefiner(expert_list=MY_EXPERT_ABILITIES, min_freq=10)),
    ('ohc', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

# Create the main preprocessor
preprocessor = ColumnTransformer(
    transformers=[
        # Your derived metrics (Calculated in a previous step)
        ('num', StandardScaler(), ['phys_bulk', 'spec_bulk', 'atk_ratio', 'stat_efficiency',
        'bst']),
        # The leak-proof ability handler
        ('abilities', ability_pipe, ['ability1', 'ability2']),
        # Types are generally safe to OHE directly as they are a fixed set
        ('types', OneHotEncoder(handle_unknown='ignore'), ['type1', 'type2'])
    ]
)

```

)

3. The Cross-Validation Loop

When you run this with `cross_val_score`, the logic works like this:

1. **Fold 1 Train:** The refiner sees that *Drizzle* appears 10 times. *Drizzle* becomes a column.
2. **Fold 1 Test:** The model predicts using the *Drizzle* column.
3. **Fold 2 Train:** In this specific subset, *Drizzle* only appears 9 times. The refiner lumps *Drizzle* into "Other." The model is forced to learn a more general pattern for this fold.

Why this is the "Gold Standard" for your project:

- **No Manual Re-coding:** You only write the `MY_EXPERT_ABILITIES` list once. The code handles the math for every fold.
- **True Generalization:** You are testing if your "Expert Choices" are actually robust across different subsets of the data.
- **Handled Unknowns:** By using `handle_unknown='ignore'` in the `OneHotEncoder`, if a rare ability shows up in your test set that wasn't "frequent" in the training set, the model won't crash—it will just treat that Pokémon as having no special ability, which is a fair and unbiased way to predict.

A Tip on the "Master List"

Since you are excluding the raw stats, your `MY_EXPERT_ABILITIES` list is your model's "eyes" into the Pokémon's utility. Don't be afraid to include abilities that "fix" bad stats, like **Huge Power** or **Speed Boost**, as these are massive predictors of a Pokémon jumping up 2 or 3 tiers.

How do you plan to compile your `MY_EXPERT_ABILITIES` list—will you go by feeling, or look at a general list of all abilities first?

2026-05-04 17:33:06

You Asked



I have question on scaling, is the standard scaler necessary considering xgboost is a tree based algorithm?

 Gemini 3 Flash

The short answer is **no**, `StandardScaler` is not strictly necessary for **XGBoost** or any tree-based algorithm. However, there are some nuanced reasons why you might still keep it or remove it.

1. Why trees don't care about scale

XGBoost works by making **binary splits** on individual features (e.g., is `phys_bulk > 5000`?).

- A tree doesn't care if a feature ranges from 0 to 1 or 0 to 1, 000, 000. The "information gain" from a split at 5, 000 remains the same regardless of the units.
- This is a major advantage over models like **Linear Regression, SVMs, or Neural Networks**, which use distance-based calculations (like Euclidean distance) or gradient descent where large-scale features can "drown out" smaller ones.

2. When Scaling can actually be helpful (Even for XGBoost)

While the algorithm doesn't need it, there are two specific reasons you might keep the scaler in your pipeline:

- **Mixed Model Benchmarking:** If you ever decide to swap XGBoost for a Logistic Regression or an MLP (Multi-Layer Perceptron) to compare results, your pipeline is already set up to handle them.
- **Convergence Speed (Marginal):** In very specific implementations of Gradient Boosting, having features on a similar scale can sometimes help the gradient calculation stabilize faster, though in 99% of Pokémon-sized datasets, you won't notice a difference.

3. When Scaling is "Bad" (Interpretability)

For your specific project, removing `StandardScaler` might actually be better for **interpretability**:

- **Feature Importance:** If you don't scale, when you look at your model's "Feature Importance" or a **SHAP** plot, the split points will make sense in "Pokémon terms."
 - *Scaled:* "The model split at `phys_bulk = 0.45` ." (What does that even mean?)
 - *Unscaled:* "The model split at `phys_bulk = 8500` ." (Ah, that's roughly the bulk of a Great Tusk!)
- **Calculated Ratios:** Your `atk_ratio` feature is naturally a small decimal. If you scale it, you lose the intuitive sense that "1.0 is the balance point."

4. My Recommendation

Remove the `StandardScaler` but keep the rest of the pipeline.

Since you are using **derived metrics** like `phys_bulk` (which will be large numbers) and `atk_ratio` (which will be small numbers), XGBoost will handle that gap perfectly fine. Keeping the raw values will make it much easier for you to "audit" your model and see exactly what physical bulk or speed tier it considers "Elite."

One exception: If you decide to use **Regularization** ($L1$ or $L2$ penalty) in XGBoost (the `alpha` and `lambda` parameters), extreme scale differences *can* occasionally interfere with how the penalty is applied across features, but even then, it's rarely a deal-breaker.

How to adjust your code: Simply remove the `StandardScaler()` from your `ColumnTransformer` and replace it with `'passthrough'` :

python

```
preprocessor = ColumnTransformer(
    transformers=[
        # 'passthrough' tells the pipeline to leave these columns exactly as they are
        ('num', 'passthrough', ['phys_bulk', 'spec_bulk', 'atk_ratio', 'stat_efficiency',
        'bst']),
        ('abilities', ability_pipe, ['ability1', 'ability2']),
        ('types', OneHotEncoder(handle_unknown='ignore'), ['type1', 'type2'])
    ]
)
```

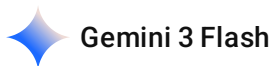
Does keeping the numbers in "Pokémon terms" make it easier for you to evaluate if the model's logic makes sense to you as a player?

2026-05-04 21:53:05

You Asked



I would like to incorporate a baseline most common class classifier



To incorporate a baseline, you want to use the **DummyClassifier** from Scikit-Learn. This is the industry-standard way to establish a "floor" for your model's performance.

In your case, since **ZU** makes up **42.27%** of your data, a "Most Common" baseline will likely have an accuracy of ~42%. If your XGBoost model isn't significantly beating that, you'll know the model is just guessing the most frequent tier.

Here is how to integrate it into your modular architecture:

1. The Baseline Code

You can run this alongside your cross-validation to get a direct comparison.

python

```
from sklearn.dummy import DummyClassifier
from sklearn.model_selection import cross_validate

# 1. Initialize the baseline (Most Frequent Class)
```

```
baseline_model = DummyClassifier(strategy="most_frequent")

# 2. Run Cross-Validation on the Baseline
# Note: Baseline doesn't need the preprocessor/feature engineering
# because it ignores the features entirely!
baseline_results = cross_validate(
    baseline_model,
    X,
    y_encoded,
    cv=skf,
    scoring=['accuracy', 'f1_macro']
)

print(f"Baseline Accuracy: {baseline_results['test_accuracy'].mean():.4f}")
print(f"Baseline F1 (Macro): {baseline_results['test_f1_macro'].mean():.4f}")
```

2. Why F1-Macro is Critical for your Baseline

Because your classes are imbalanced (19 Ubers vs. 194 ZU), **Accuracy** is a deceptive metric.

- **Accuracy:** The Baseline will get ~42% because it gets every ZU Pokémon right and everything else wrong.
- **F1-Macro:** The Baseline will have a **very low** F1-Macro score (likely around 0.07 - 0.10). This is because F1-Macro treats the "Ubers" class with the same importance as "ZU." Since the baseline **never** predicts Ubers, its F1 for that class is 0.

Your XGBoost model needs to show a massive improvement in **F1-Macro** specifically to prove it has actually learned the difference between an Uber and a ZU.

3. Comparing the Two

I recommend creating a small helper function to print the comparison. This makes your "Success" clear:

Metric	Baseline (Most Frequent)	XGBoost Model
Accuracy	~0.4227	Your Result
F1-Macro	~0.0860	Your Result

A Note on Stratified Splits

Since you are using `StratifiedKFold`, your baseline performance will be extremely stable across folds because every fold is guaranteed to have the same ~42% ZU ratio.

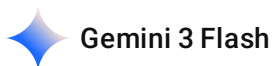
One final tip: Since your baseline is "Most Frequent," it will effectively predict "ZU" for every single Pokémon. If your XGBoost model's **Confusion Matrix** shows that it is also predicting ZU for

"borderline" Pokémon (like RU/NU), you'll know you need to adjust your class weights in XGBoost (`scale_pos_weight` or `sample_weight`) to make the model "care" more about the rarer, higher tiers.

Would you like to see how to generate a **Confusion Matrix** at the end of your cross-validation to see exactly which tiers your model is confusing with ZU?

2026-05-04 21:59:36 You Asked 

how can I add f1 score to my current xgboost code



To add the **F1 Score** to your XGBoost code, you need to update the `scoring` parameter in your `cross_val_score` (or `cross_validate`) function.

Since you have 8 different tiers, you should use `f1_macro` . This calculates the F1 score for each tier independently and then takes the average, ensuring that "Ubers" (with only 19 samples) is just as important to the final score as "ZU" (with 194 samples).

1. Simple Implementation (`cross_val_score`)

If you just want the F1-Macro numbers:

```
python

from sklearn.model_selection import cross_val_score

# Run CV focusing on F1-Macro
f1_results = cross_val_score(
    pipeline,
    X,
    y_encoded,
    cv=skf,
    scoring='f1_macro'
)

print(f"Mean F1-Macro Score: {f1_results.mean():.4f}")
print(f"Std Dev of F1-Macro: {f1_results.std():.4f}")
```

2. Comprehensive Implementation (`cross_validate`)

If you want to track **both** Accuracy and F1-Macro at the same time (highly recommended for imbalanced Pokémon data), use `cross_validate` :

python

```
from sklearn.model_selection import cross_validate

# Track multiple metrics
metrics = ['accuracy', 'f1_macro']

results = cross_validate(
    pipeline,
    X,
    y_encoded,
    cv=skf,
    scoring=metrics,
    return_train_score=False
)

print(f"Mean Accuracy: {results['test_accuracy'].mean():.4f}")
print(f"Mean F1-Macro: {results['test_f1_macro'].mean():.4f}")
```

3. Adding F1 to XGBoost's Internal Training

If you want XGBoost to actually "watch" the F1 score during training (to prevent overfitting), you can add it to the `fit` parameters within the loop. However, since you are using a **Pipeline**, you usually pass these through the `classifier` step.

Note that XGBoost's native multi-class F1 evaluation is slightly complex, so for your size of data (459 points), the Scikit-Learn `cross_validate` approach above is the standard and most reliable method.

Why F1-Macro is your "North Star"

In your specific dataset:

- **Accuracy** is "The ZU Metric." If the model predicts ZU correctly but fails at everything else, Accuracy will still look okay (~40%).
- **F1-Macro** is "The Competitive Metric." If the model fails to identify **Ubers** or **OU** Pokémon, the F1-Macro score will tank.

Pro-Tip: If your F1-Macro is significantly lower than your Accuracy, it means your model is "lazy" and is over-predicting the most common class (ZU). If this happens, you can add `sample_weight` to the `fit` method to force the model to pay more attention to the rarer tiers.

Does your current XGBoost model show a big gap between Accuracy and F1-Macro?